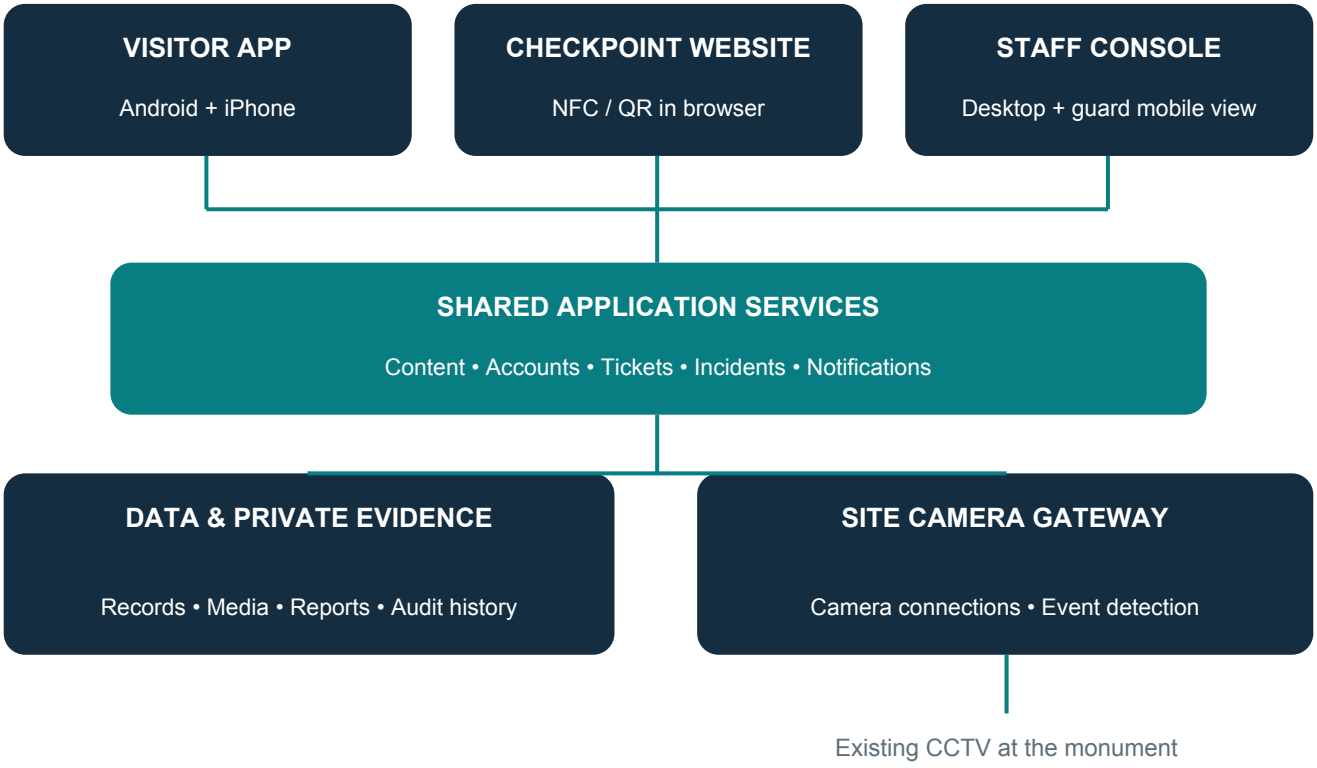
Product vision, current status & PRD

A connected heritage experience for visitors, site teams and cultural institutions.

One complete reference for what Sanskriti is, how the finished platform will work, what exists today, and what must be built next.



Document status
Planning baseline v1.1 • 30 September 2026
Current-state findings are separated from proposed features. Requirements and timelines are proposed until accepted by the team and pilot site.

Includes: product experience • dashboard structure • verified progress • future roadmap • iOS completion PRD • monument operations PRD

READING GUIDE

How to use this document

Part	Contents	Pages
01 / Product vision	Purpose, users, app structure, dashboards and connected journeys	3–9
02 / Current position	Implemented source, verified iOS build, gaps and delivery roadmap	10–13
03 / Product requirements	Scope, iOS requirements, operations requirements and acceptance gates	14–24
04 / Delivery reference	Execution backlog, decisions, glossary and evidence	25–26

Three labels used throughout

• **Implemented in source:** code or assets exist. This does not automatically mean the feature is deployed, tested or ready for customers.

• **Verified in this review:** an explicit check was completed, such as the unsigned iOS simulator build.

• **Proposed:** intended behavior or a future requirement. It still needs implementation and acceptance.

What a PRD means

A Product Requirements Document states who a feature serves, what it must do, what is outside its first release, and the evidence needed to call it complete. The PRD sections below turn the vision into a delivery agreement.

The central distinction
The visitor product already has substantial implementation. Monument operations is a new product track. The existing marketplace administration screen is not a monument control room.

This document is intentionally understandable without a coding background. Technical terms are explained where needed; implementation detail supports the product decisions rather than replacing them.

What Sanskriti is

Sanskriti connects two needs: helping people understand and enjoy heritage, and helping the people responsible for a heritage site manage and preserve it.

For a visitor

A person can discover a monument, identify it with a camera, understand its history through Yatrik, follow an exploration route, play a Treasure Hunt, and discover local crafts. Someone arriving for a single visit can use a QR or NFC link without installing the app.

For a site team

A manager can see operational activity, staff can validate admission, a supervisor can review reports and camera alerts, and a guard or conservation officer can receive an assignment and record the outcome.

The shared value

The same monument identity connects its visitor stories, physical checkpoints, staff zones and incident records. Public content and private operations remain separated by permissions even though they refer to the same place.

One example
A visitor scans a plaque and starts a story. They notice damage and submit a photo. The site supervisor verifies it, assigns a conservation officer and records the response. The visitor experience and the preservation workflow connect through the monument and zone.

What success looks like

- Visitors can complete an engaging, understandable journey with minimal friction.
- Every confirmed operational issue has an accountable owner and recorded outcome.
- A site can measure benefits well enough to decide whether to keep using the service.

Who uses which interface?

Person	Where they work	What they need
Visitor / traveller	Android or iOS app	Discovery, recognition, Yatrik, exploration and saved progress
One-time on-site visitor	Mobile checkpoint website	Stories, audio, Hunt and reporting without installation
Monument manager	Operations console	Site overview, staff, incidents, reports and configuration
Control-room supervisor	Operations console	Camera health, suspected alerts, verification and escalation
Guard / field staff	Mobile view of operations console	Assigned work, directions, acknowledgement and resolution
Ticketing staff	Restricted admission screen	Ticket validation and entry/exit exceptions
Conservation officer	Operations console / mobile view	Damage cases, maintenance tasks and evidence
Artisan / seller	Separate artisan workspace	Application, listings, inventory, workshops and bookings
Organisation administrator	Multi-site administration view	Monuments, authorised staff and organisation-level reports
Sanskriti administrator	Platform administration	Customer organisations, platform configuration and support

These are role-specific views. They do not all require separate mobile applications. The guard view can be a responsive website, and organisation administration can live inside the same staff console.

The completed visitor app

Recommended navigation: five clear tabs, with the monument detail page acting as the centre of each visit. This is a proposed final organisation, not a claim about the current navigation.

Tab	Main contents
Home	Search, nearby and featured monuments, festivals, suggested journeys and access to local crafts
Explore	Map, safe checkpoint routes, camera-assisted guidance, narration and historical scenes
Scan	Take or choose a photo, identify a monument, inspect confidence and open its detail page
Yatrik	Text and voice questions, monument context, narration and language selection
My Journey	Saved places, itineraries, tickets/bookings, Hunt progress, badges, rewards and profile

Inside a monument page

Overview → history and architecture → visitor information → Explore / audio / Yatrik → Hunt and quiz → historical scenes → nearby crafts and workshops → report an issue.

How features connect

Recognition selects a monument; that selection gives Yatrik context and opens the relevant route, stories and quiz. Completing a Hunt updates saved progress. Marketplace bookings connect to the visitor account. A voluntary issue report links to the site team, without exposing internal staff records.

Release boundary
The full vision may include richer historical personas, broad language coverage and production commerce. The first completed iOS release should focus on dependable core visitor journeys rather than making every future module a release dependency.

The NFC / QR visitor website

NFC means tapping a phone on a small tag. QR means scanning a printed code. Both open the same secure web page for a specific monument checkpoint.

Step	What the visitor sees
1. Tap or scan	Monument name, checkpoint identity and language selection
2. Choose an experience	Explore, listen to a story, or begin a Treasure Hunt
3. Visit a checkpoint	Short content, optional audio, a clue/question and safe next-step directions
4. Continue	Progress saved on the device, with a useful weak-network state
5. Finish	Completion acknowledgement; optional sign-in to retain longer-term progress
At any point	Facilities/accessibility information and a voluntary issue-report option

What is shared with the mobile app

Monument identity, reviewed stories, route/checkpoint content, language versions and compatible progress records. The website is a shorter on-site experience; advanced navigation remains in the app until browser behavior is proven.

What staff manage

The content/checkpoint dashboard publishes stories and clues, updates a destination link, disables a damaged plaque and measures starts/completions. A content update must not break an ongoing session.

First pilot
One monument • Five physical checkpoints • One Explore route • Five-stage Hunt • Hindi and English content • One completion reward. The document is English-only; the proposed pilot product supports both languages.

The monument staff console

Section	What happens here
Overview	Open incidents, urgent assignments, zone crowd levels, admissions and camera health
Zone crowd map	Active-user signals, camera estimates, confidence, congestion alerts and safer route suggestions
Visitors & tickets	Validate authorised tickets, record admission/exit and explain rejected or duplicate scans
Cameras & alerts	Inspect permitted feeds/clips, camera availability and AI-suspected events
Incidents	Review visitor reports and alerts, confirm/dismiss, assign staff and record resolution
Conservation	Turn verified damage into maintenance work with owner, due date and before/after evidence
Staff	Invitations, permitted sites, roles and operational assignments
Content & checkpoints	Reviewed stories, routes, translations, Hunt clues and NFC/QR destinations
Analytics & reports	Engagement, admissions, confirmed incidents, response times and unresolved work
Settings	Site zones, cameras, ticket rules, notification preferences and retention

The guard view stays simple

My assignments → incident details → acknowledge → start action → add notes/photo → submit resolution. Guards should not need the desktop camera wall or organisation configuration screens.

Different views, shared records

When a supervisor assigns an incident, it appears in the guard view. Updates return to the same incident record and the manager's overview. There should not be separate, conflicting copies of the same case.

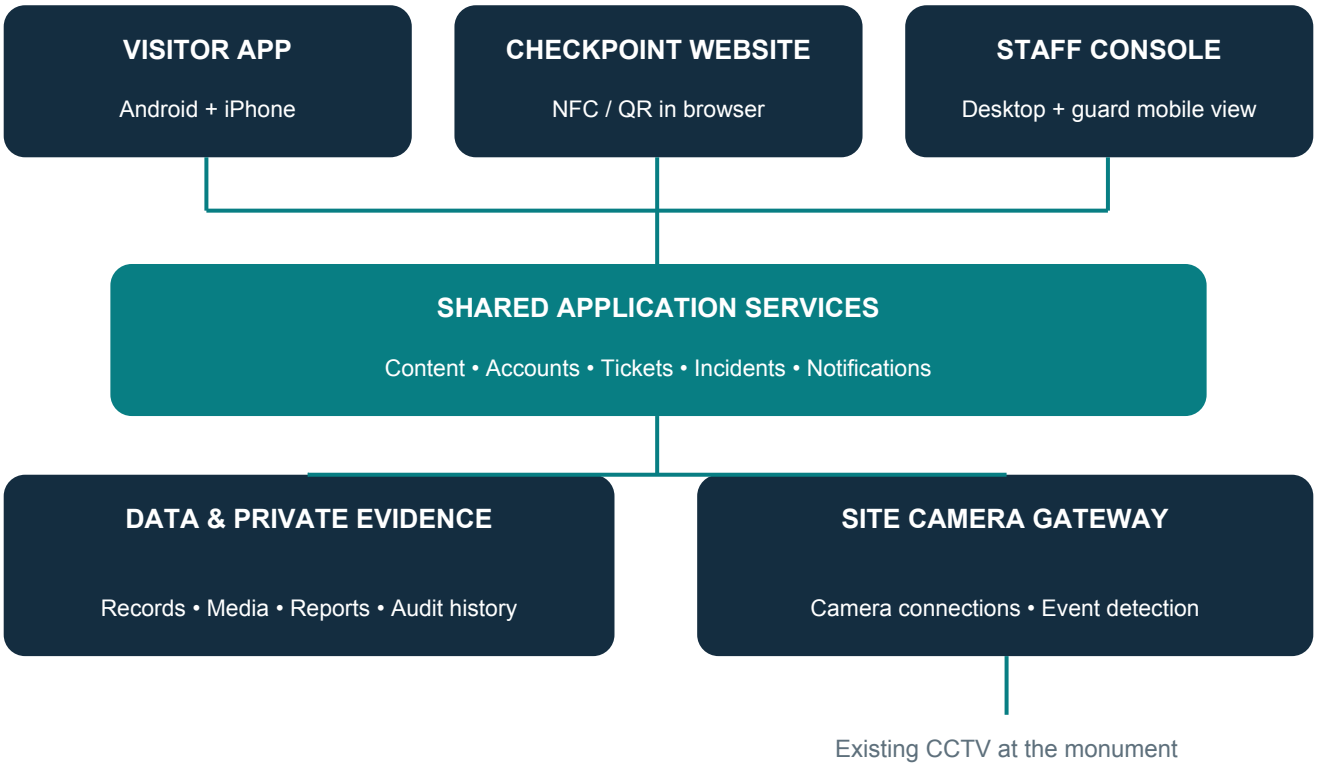
Other dashboards and shared services

Artisan and marketplace administration

Artisans manage their applications, approved listings, stock or workshop capacity and reservations. Marketplace reviewers handle moderation and fulfilment oversight. These permissions are separate from camera evidence, guards and monument incidents.

Organisation and platform administration

An organisation administrator manages its own monuments and staff. A Sanskriti platform administrator manages customer setup and platform support. Cross-customer access must be explicitly controlled and recorded; being a marketplace administrator must not grant operations access.



The backend is the shared service layer behind the screens. It checks permissions, stores records and coordinates work. The on-site camera gateway connects CCTV and sends relevant events/evidence; it is not the visitor's phone camera.

A complete connected journey

Event	System connection	Who acts next
Visitor discovers a monument	One monument identity links content, map, stories and route	Visitor
Visitor taps checkpoint plaque	Checkpoint opens the correct content and records an anonymous session	Visitor
Visitor notices damage	Submitted photo/category/zone becomes an unverified report	Supervisor
Camera notices restricted entry	Gateway sends a suspected event and a short evidence record	Supervisor
Supervisor checks evidence	Confirm, dismiss or link a duplicate report; set priority	Supervisor
Confirmed issue is assigned	Same incident appears in the permitted staff member's queue	Guard or conservation officer
Staff records action	Notes, timestamps and evidence update the incident	Supervisor / manager
Case is closed	Outcome appears in daily reports and operational analytics	Manager

Two boundaries that matter
AI proposes an alert; a human verifies it. Ordinary visitor camera use supports recognition/navigation; reporting evidence is a separate, voluntary submission.

If internet access fails, the visitor sees whether a report is only saved locally or has actually been delivered. The camera gateway can retry queued events. Staff must not interpret missing data as "everything is safe".

02 / CURRENT POSITION

What already exists

The repository contains a substantial Android/shared-UI foundation. The table describes implementation presence, using the existing project report and source inspection; no fresh Android acceptance run was performed for this document.

Area	Current position
Android and shared interface	Android project, bundled UI and native integration source exist. Release readiness is a separate check.
Visitor experience	Discovery, monument pages, recognition/chat paths, Yatrik, narration, Explore, Hunt, quiz, rewards and supporting screens exist.
Historical content	Era media and related presentation exist; depth, factual review and richer persona experiences still need work.
Marketplace	Screens, local demonstration flows, data migrations and service functions exist. Production payments/fulfilment are not verified here.
iOS	Xcode project, shared-interface wrapper and Apple service scaffolding exist. Unsigned simulator compilation was verified.
NFC/QR website	Planned. The standalone checkpoint product is not established as completed.
Monument operations	Planned. Marketplace admin is not the management console.
CCTV and operational AI	Planned. Site integration, model evaluation and staff workflow remain to be built.

What “done” must mean

A visible screen is not enough. A completed feature needs working services, content, permissions, failure handling and a passed user journey on the intended device.

02 / CURRENT POSITION

iOS: the verified baseline

Current classification: buildable hybrid prototype; feature parity and distribution readiness remain unverified. “Hybrid” means the iPhone app displays the shared web interface while native Apple code supplies device capabilities.

Finding	Meaning
Fresh simulator build succeeded	The current code compiles as an unsigned Debug simulator app. This does not prove camera, location or live AI behavior.
Active screen is WKWebView	The root loads the shared interface. Separate Swift screens are mostly scaffolding, not the active visitor experience.
Xcode copies an existing UI bundle	Packaging uses the Android asset folder, approximately 68 MB at inspection. It does not automatically rebuild editable web source.
Local server serves files	The server cannot execute server API routes. Relative /api requests need an explicit service connection.
Android bridge has no iOS equivalent	The displayed interface is not wired to the Swift services through an iOS message bridge.
Local profiles and demo commerce	Bundled-mode logic uses local profiles and demonstration marketplace paths.
Tests are not connected	A navigation test file exists, but the project has no configured test target.

Additional release work: persistence across relaunch, startup recovery, permission handling, device testing, icon/privacy declarations, signed archive and TestFlight. The local server chooses a changing port, which creates a storage-origin risk to reproduce and correct.

What remains and what is uncertain

iOS completion is primarily integration work

The shared screens provide a starting point. The critical work is making those screens reliably talk to the intended backend, device hardware and persistent storage, then proving that the complete journeys work on iPhones.

Known gap	Next evidence required
Mixed API paths and local-only routing	Installed app completes real question/photo requests against the intended server
Missing iOS device adapter	Capture, recording, location and narration pass success/denial/cancellation checks
Persistence risk	Profile, language and Hunt progress survive force quits and a bundle update
Unconnected tests and release setup	Automated test target, physical-device checklist and signed TestFlight installation
Management product not implemented	One real report can be reviewed, assigned, resolved and included in a report

What this review did not verify

No physical-iPhone journey, live AI request, deployed database state, payment, camera feed, signed archive or TestFlight release was verified. No simulator runtime session was executed as part of the build check.

No invented percentage
There is no defensible “iOS is 80% complete” claim yet. Measure progress by passed release requirements: packaging, services, device integration, persistence, journey testing and distribution.

How to reach the full vision

Phase	Outcome	Exit gate
A / Finish core visitor quality	Complete iOS integrations; improve reviewed content and existing Android reliability	Core journeys pass on real devices
B / Operational foundation	Staff roles, visitor reports, review/assignment, guard view and daily report	Report-to-resolution works without AI
C / Connected on-site pilot	Five checkpoints, visitor web journey and authorised ticket validation	Visitor flow and admission exceptions work at one site
D / Camera pilot	One compatible camera first, then 4–10; two detection categories	Measured alert quality and manageable review workload
E / Controlled expansion	Conservation depth, multi-site views, more monuments/languages and validated commerce	Repeatable setup and customer value demonstrated

Planning assumptions

A 90-day platform pilot assumes three engineering owners covering mobile/shared UI, backend/web and video integration, plus part-time QA/content and a site coordinator. These are role needs, not a claim about the current team. A team learning through AI tools should budget for experienced technical review and a longer calendar.

The iOS estimate is approximately 4–6 weeks with an experienced mobile engineer and backend support after scope and service access are settled. The platform and iOS can overlap, but should not share an unrealistic single-person deadline.

Dependencies
iOS depends on shared services and phone testing. CCTV depends on site access, compatible cameras and labelled evaluation footage. Neither should block the manual incident workflow.

Scope and delivery agreement

Product objective: deliver a dependable visitor experience and a site operations workflow that converts reports or suspected events into accountable human action.

Release	Must include	Outside the initial commitment
iOS core release	Guest/local onboarding, monument content, recognition, Yatrik, narration, Explore, Hunt and saved progress	Fully native screen rewrite; production commerce unless separately accepted
Operations pilot	One site; staff access; reports; verification/assignment; guard view; audit history; daily report	Large-scale rollout, government sales replacement or predictive staffing
Connected pilot	Five checkpoints; Hindi/English; ticket validation; compatible camera gateway; two evaluated categories	Broad unsupervised incident detection or autonomous enforcement

Priority language

P0: required for the relevant release gate. **P1:** next phase after the core works. **P2:** later expansion with separate validation. Priority is scoped to a release, not an instruction to build everything simultaneously.

Ownership and approval

The product owner accepts scope and wording. The technical owner accepts architecture and implementation quality. QA records acceptance evidence. The site manager accepts operating procedures. No requirement is complete until its named owner can show its acceptance result.

Requirement format

Every requirement below states the intended behavior, the practical implementation approach and the acceptance check. The team should turn each ID into a tracked task and attach evidence before closing it.

I1–I3: connect the active app

I1 / P0 — Repeatable app packaging

Requirement: Every build must contain the correct shared interface and a known content version.
How: Establish one export/build step used by Android and iOS, record revision and asset version, and fail when required files are missing. Update documentation to describe the hybrid approach.
Accept: A clean checkout produces the app without manually reusing an old Android export. Launch errors show a retry path rather than a blank screen.

I2 / P0 — Consistent online services

Requirement: Released screens must use one configured HTTPS service contract.
How: List every active request and its response, reconcile legacy chat paths with current routes, provide missing recognition/transcription endpoints, and remove reliance on local static /api execution. Keep provider credentials on the server. Ship non-secret production configuration that works without Xcode.
Accept: An installed build sends a real question and image, displays valid results, handles timeout/offline states and contains no provider secret.

I3 / P0 — iPhone device capabilities

Requirement: Camera/photo selection, audio recording, location/heading and narration must work from the displayed interface.
How: Add an iOS adapter alongside Android/browser adapters. Use a narrow message bridge where native support is needed; validate operation, origin, parameters and request ID. Support cancellation and errors.
Accept: Every required capability passes allowed, denied and cancelled scenarios on a physical iPhone. Unsupported capabilities show a useful alternative.

Implementation principle
Complete the active hybrid experience first. Do not count unconnected Swift screens as completed app features or begin a full native rewrite without a separate product decision.

I4–I6: make it dependable

I4 / P0 — Saved state and lifecycle

Requirement: Progress must survive app restart and device interruptions.
How: Reproduce the changing-localhost-port storage issue. Use app-owned persistent storage where needed, protect authentication secrets in Keychain and migrate existing data. Recover loading/WebKit failures, restrict the local listener to loopback, and stop sensors/audio on exit.
Accept: Profile, selected language and Hunt progress survive ten force-quit/relaunch cycles and a bundle update. Background/resume does not duplicate recording or navigation sessions.

I5 / P0 — End-to-end visitor journeys

Requirement: Every core release feature must have a complete success and failure journey.
How: Test recognition with a monument and unrelated photo; voice with English/Hindi and interruption; navigation outdoors with weak GPS; Hunt with resume and duplicate completion; offline stories in airplane mode. Fix keyboard, safe-area and readable-text behavior.
Accept: The agreed matrix passes on at least two physical iPhones, including the minimum supported OS where available. No unresolved release-blocking defect remains.

I6 / P0 — Test and distribution readiness

Requirement: The app must be testable and installable without the development environment.
How: Connect an XCTest target and scheme; test bridge errors, API parsing and persistence. Prepare Release build, signing, icon, screenshots, support/privacy information and applicable privacy declarations. Upload through the authorised Apple account.
Accept: A signed TestFlight build installs and completes the core journeys without Xcode. App Store approval remains a separate external decision.

Apple guidance used: WKWebView permission delegation, privacy manifests, submission preparation and TestFlight. Links are included in the evidence reference on page 26.

M1–M2: accounts and accountable action

M1 / P0 — Site identity and staff permissions

User need: Staff work only on the sites and actions they are authorised to access.
How: Create organisation/site membership, invitations, roles and zones. Keep visitor identity and marketplace permissions separate. Check permissions in shared services and database rules, not only on screens. Record role changes and access to sensitive evidence.
Accept: A valid user from site A cannot read, modify or subscribe to site B records. Removing membership blocks operational access. Guards cannot grant themselves manager privileges.

M2 / P0 — Incident review and response

User need: A supervisor can turn a report into assigned work and prove what happened.
How: Implement the state sequence below, require an owner for confirmed work, log transitions and define priority escalation. Give guards an acknowledgement/action/resolution view. Link duplicate reports rather than creating conflicting cases.

Incident state model
Reported → Under review → Dismissed OR Confirmed → Assigned → Acknowledged → In progress → Resolved → Closed

Reopening requires a reason. Resolution and final closure may be separate permissions. AI alerts begin as suspected events, not confirmed incidents.

Accept: A manual report reaches closure with reviewer, assignee, evidence, timestamps and outcome. Retries do not create duplicate incidents. An overdue priority case reaches the supervisor through the agreed notification channel.

Build first: Use manually entered cases before connecting cameras. This proves the operational workflow independently of detection quality.

M3–M4: visitors and checkpoints

M3 / P0 — Voluntary visitor reporting

User need: Visitors can report a site issue without gaining access to staff information.
How: Provide category, checkpoint/zone, optional photo and a preview-before-submit step. Allow location correction, limit file size/type, verify upload completion and rate-limit abuse. Distinguish locally saved drafts from delivered reports. Route the submission to the correct site queue.
Accept: A delivered report appears for authorised reviewers and remains unverified until reviewed. Visitors cannot read other submissions or private evidence. Offline drafts show their delivery status and retry without duplication.

M4 / P0 — Public checkpoint experience

User need: A visitor can complete a guided journey without installing an app.
How: Use stable HTTPS links for NFC and QR, choose language, offer anonymous Explore/Hunt, cache essential content and save progress. Give staff a draft/review/publish workflow and version content so changes do not break active visits.
Accept: All five checkpoints work in Safari and Android Chrome in both pilot languages. A temporary network failure preserves progress. Repeated requests cannot issue the same reward twice.

Content record required for each checkpoint

Site and zone • stable checkpoint ID • safe location/order • title/story • audio/media • language versions • clue/answer rule • source/reviewer • publication version • active/retired state.

Trust boundary
Static NFC/QR links can be copied. A scanned link is not proof of physical presence. Valuable rewards or official attendance need a stronger separately validated check.

M5: admission and crowd visibility

M5 / P0 for connected pilot — Ticket validation

User need: Ticketing staff receive a clear, reliable decision about admission.
How: Integrate an authorised existing ticket source through approved import/API access. Define valid, already used, expired, cancelled, wrong-site and wrong-slot outcomes. Make checking and recording admission one indivisible operation so two scanners cannot accept the same ticket at once.

Define re-entry and exits. Log manual overrides with staff identity and reason. Agree an offline procedure with the site; a disconnected scanner must not silently report an unverified ticket as valid.

Accept: Concurrent duplicate scans result in one accepted admission. Each rejected scan has an understandable reason. Overrides are auditable and offline behavior follows the agreed procedure.

Occupancy is an estimate with a counting method

Starting occupancy + accepted entries − recorded exits gives one estimate. Missing exits, gate coverage and re-entry rules affect its quality. Camera counts and consented active-user zone signals add local visibility. Keep all sources labelled; the zone-management requirement on the next page defines how to combine them without double-counting.

Accept: Staff can see stale or incomplete data, reconcile a count and explain why the dashboard value changed. No exact crowd number is shown when the underlying data cannot support it.

Scope boundary
Validate existing authorised tickets first. Replacing government ticket sales, obtaining provider access, and handling new commercial ticket payments are separate integrations, not assumed MVP capabilities.

M5A: zone-wise crowd management

P0 for the connected crowd pilot / Proposed, not implemented. Give managers a zone map showing where visitors are gathering, how reliable the estimate is, and when intervention is needed.

Three inputs, with different meanings

Input	How it is collected and used
Active-user signal	With location consent, map recent app location updates to site-defined zones. Recent checkpoint interactions provide a weaker presence clue. Count each session once and expire stale presence; an open app alone does not establish location.
Camera estimate	Count people in calibrated camera zones, including non-app users. Track camera health and account for overlapping coverage. Label obscured, stale or unavailable views.
Entry/exit baseline	Use accepted admission and recorded exit events to check the site-wide total. This cannot locate people within individual zones.

How the combined estimate works

Use fresh, calibrated camera counts as the initial count estimate where coverage is adequate. Show active-app presence separately to explain movement and emerging demand. Do not add the two counts: they can describe the same people. Compare with entry/exit totals to flag discrepancies. Where camera coverage is missing, label app counts as a partial sample; do not extrapolate to all visitors without field validation.

Dashboard and response

For each zone show estimated occupancy or range, active participating users, configured capacity, last update, source coverage and confidence. Use low/moderate/high crowd labels based on site-approved thresholds, sustained duration and reset rules. Alert staff for sustained congestion. Suggest a less-crowded route only when it is open, accessible and safe; staff can override it.

Implementation and acceptance

Define zone boundaries, freshness windows and consent controls. Test boundary drift, overlapping cameras, stale phones, app closure, denied location and camera outages. Calibrate against manual counts and agree count-error and false-alert limits before activation. Accept only when duplicate users/camera coverage are handled, stale data is visibly degraded, and an alert reaches the right staff member. Opting out must preserve the core visitor experience; show aggregate crowd data, not individual visitor trails.

M6–M7: cameras and reviewed AI

M6 / P0 for camera phase — Camera gateway

User need: Authorised staff can inspect relevant evidence and know when a camera is unavailable.
How: Test one compatible feed; store credentials on the gateway; add heartbeat, reconnect/backoff, timestamps and bounded local buffering. Send authenticated events with unique IDs and retry safely. For a live camera wall, provide an authorised browser-compatible streaming layer rather than assuming raw camera feeds play in a browser.
Accept: Camera disconnection appears as offline; queued events arrive once after reconnection; clips are restricted to authorised staff. Expand to 4–10 feeds only after measuring capacity.

M7 / P0 for camera phase — Two detection categories

User need: Supervisors receive useful suspected alerts without being overwhelmed.
How: Begin with calibrated crowd thresholds and restricted-zone crossing/dwell. Mark zones on each camera view; evaluate detector/tracker licensing and footage suitability. Use labelled examples and separate evaluation footage; include occlusion and different lighting. Add duplicate suppression and cooldowns.
Accept: Observation-only evaluation meets agreed precision and false-alert burden for each category. Report missed events where labelled reference evidence exists. A person confirms or dismisses every alert before action.

Not in the pilot
Facial identification, autonomous punishment and broad claims of reliable “vandalism detection”. Later categories need their own definition, data and evaluation.

M8–M10: preservation and management

M8 / P0 basic, P1 depth — Reports and conservation

How: Show open/overdue cases, admissions, occupancy quality and camera health. Generate a site-timezone daily report separating suspected, dismissed and confirmed events. Convert verified damage into a maintenance task with owner, due date and evidence. Add approval and before/after records for conservation closure.
Accept: A manager can trace report totals back to records. Missing data is not reported as zero. Maintenance completion has an accountable owner and supporting evidence.

M9 / P0 — Staff configuration and notifications

How: Provide site/zone setup, staff invitations and assignments, checkpoint/camera configuration and ticket rules. Start with in-app notifications and add another channel only after the site selects it. Define staffed hours, escalation recipients and overdue thresholds.
Accept: Configuration changes are recorded; reassigned work appears for the correct staff member; priority escalation reaches the agreed supervisor without repeated duplicate notifications.

M10 / P1 — Organisation and expansion views

How: Add multi-site summaries after single-site isolation works. Show comparable metrics with consistent definitions. Allow organisation administrators to manage their authorised sites without exposing another customer. Keep artisan and marketplace administration in a separate permission area.
Accept: Aggregated totals reconcile to permitted sites, and user access is tested across two organisations before rollout.

Full-product additions
Richer conservation scheduling, larger language/content coverage, production marketplace operations and predictive staffing belong to later scoped releases after the pilot proves value.

Data, reliability and release quality

Requirement	Implementation and acceptance
Shared identities	Use stable organisation, site, zone and checkpoint IDs. Map existing heritage-site records before adding new ones. The same monument must not become conflicting records across products.
Controlled access	Enforce site membership in services and database policies. Keep provider/admin secrets server-side. Private evidence is available only through authorised, time-limited access.
Reliable retries	Use unique request/event IDs for reports, admissions, gateway events and rewards. Repeating a request must not repeat its real-world effect.
Content quality	Record sources, reviewer, language and version. Label reconstructions and legends. Validate safe routes and accessible alternatives before publishing.
Failure behavior	Display offline/stale/delayed states. Queue only actions safe to retry. Never imply that an undelivered report has reached staff.
Operational recovery	Use separate testing and live environments, monitored services, backups and rollback. Rehearse restoration and gateway/internet outages.
Evidence policy	Agree collection, retention, deletion and staff access with the pilot site. Record evidence access; collect only what the workflow needs.
Usability	Test readable text, keyboard/focus, labels, contrast and small screens. Visitor and guard tasks must be usable without training in the software.

Supabase is the proposed shared data/auth foundation already represented in repository source. Deployment state was not inspected. Row-level security means the database itself limits which records an account can access; visible dashboard controls alone are insufficient.

When can we call it complete?

Gate	Required evidence
iOS beta	Clean packaging; working online calls; device capabilities; retained progress; real-phone journey results; signed TestFlight installation
Manual operations	Site/role isolation; report-to-closure record; duplicate prevention; escalation; useful daily report
Visitor pilot	Five functioning checkpoints; both languages; Safari/Chrome checks; resumable journey; source-reviewed content; consented zone presence
Camera activation	Feed compatibility; health/outage handling; labelled evaluation; zone-count calibration and overlap checks; acceptable alert workload; trained human review
Pilot completion	Documented outcomes, unresolved limitations, cost/support evidence and a buyer decision on continuation

Proposed measurement targets, not existing results

Visitor targets: at least 30% of eligible scan sessions start an experience; at least 20% of starts finish the Hunt; median three checkpoints; fewer than 10% fail because of network/browser problems; average collected satisfaction at least 4/5. Fix session and completion definitions before measuring.

Operations targets: priority acknowledgement within two minutes during staffed hours; at least 80% reviewed precision per chosen category; a site-agreed false-alert limit per camera-hour; owner and history for every confirmed incident. Agree evaluation duration, sample size and missed-event measurement before activation.

Commercial gate

Separate setup costs from recurring hosting, AI, evidence storage and support. Earlier ranges of INR 3–8 lakh for a pilot and INR 6–18 lakh annually per site are unvalidated hypotheses. Obtain workload evidence and site-specific costs before committing to a price.

Execution order and open decisions

Window	Work to finish	Primary owner
Days 1–15	Freeze iOS scope; inventory APIs; select site; document roles and incident prototype	Product + mobile + backend
Days 16–30	Device/persistence fixes; staff accounts; reports and assignments; checkpoint prototype	Mobile + web/backend
Days 31–45	iOS beta gates; ticket validation; zone definitions/consented presence; one-camera connection	QA + backend + video
Days 46–60	Beta fixes; gateway retries; zone-count/category calibration; full alert-to-resolution flow	Mobile + video + site team
Days 61–75	Install checkpoints; expand compatible feeds; train staff; observation-only analytics	Site coordinator + video + QA
Days 76–90	Controlled activation; weekly review; outcomes and commercial decision	Product + site manager

Decisions to record before committing delivery dates

- Who owns each engineering track, and who provides experienced technical review?
- Does the first iOS release require cloud accounts, or is local/guest progress enough?
- Which backend is authoritative, and which endpoints remain to be implemented?
- Which pilot site grants access to staff, ticket examples, cameras and installation?
- Which rewards, offline admission rules, evidence retention and support hours are agreed?

If the same person owns multiple tracks, extend the schedule or reduce scope. Preserve the sequence: working manual workflow first, camera intelligence after it.

Glossary and evidence

Term	Plain-language meaning
Backend / API	The shared services that screens call to read information or perform authorised actions
Hybrid app / bridge	A shared web interface inside a mobile app; a bridge connects it to phone capabilities
Gateway	A site-side component that connects cameras and sends events/evidence
Role / site isolation	Controls defining who can do what, and which monument's records they can access
Acceptance gate	A specific set of checks that must pass before a release or pilot advances
TestFlight	Apple's distribution channel for testing iPhone app builds before public release

Project evidence

Current status is based on the Sanskriti repository: iOS app entry, Views, WebView, Services and DeviceServices; Xcode project/scheme and test file; mobile export script; shared API/device/auth/marketplace clients; and the existing project report. The detailed implementation review is saved as *output/project-report/implementation-plan-2026-09-30.md*.

The fresh unsigned Debug simulator build completed successfully on 30 September 2026 using installed Xcode 27.0. Log: /tmp/sanskriti-ios-plan-build.log. No physical-device or production-service acceptance is claimed. File presence and source inspection are distinct from verified runtime behavior.

Official implementation references

Apple: web capture permission handling

Apple: privacy manifest preparation

Apple: app submission preparation

Apple: TestFlight overview

Supabase: database row-level security

These sources informed the implementation recommendations; they do not certify this app. Product structure, priorities, staffing and estimates in this document are proposed planning decisions.